Mojo ownership cheat sheet

Who owns a value, and how it passes between functions.

v1.2

THE MODEL

Value ownership is fundamental to Mojo. Every value has exactly one owner, and how values move between owners runs through the whole language.

You encounter ownership in two situations: *variables* and *function calls*.

Variables can own or reference a value. *Argument conventions* describe how a function uses a value: read-only, reference, mutable, owned, produced, or consumed.

VAR AND REF

VAR OWNS, REF REFERS

```
var data: List = [1, 2, 3] # owns the list
ref view = data[0] # a 2nd name, no copy
view = 9 # writes through to data
print(data) # [9, 2, 3]
```

`var` always means "I own this." `ref` means "this is a view into someone else's value." A struct's `var` field owns its value and a struct type owns its fields.

VAR ASSIGNMENT

What happens when you assign a var binding

A `var` assignment uses the right-hand side's policy: it determines whether the value is materialized, constructed, copied, or transferred. A call or expression returning a value returns a transferrable value.

YOU ASSIGN	THE VAR TAKES OWNERSHIP OF
`5.0 / "Hello" / [1, 2, 3] (a literal)`	a materialized literal
`SomeType()`	a freshly constructed value
`some_value (ImplicitlyCopyable)`	an implicit copy
`some_value.copy() (Copyable)`	an explicit copy
`some_value^`	the source's value, transferred
`some_ref`	a copy of the referenced value

A copy doesn't change a value's ownership. Only the `^` sigil enables transferring a value to a new owner.

REFS AND ORIGINS

ORIGINS ON REFERENCES

```
def first[T: Movable](ref xs: List[T]) ->
ref[xs[0]] T:
    return xs[0]

ref x = first(xs) # len(xs) known to be > 0
```

A `ref` return carries an `origin` so the compiler tracks where it points, whether it stays valid, and whether access is mutable. Values are destroyed at last use; a live `ref` keeps the value it refers to alive.

MUTABILITY

Values aren't mutable or immutable. Access is.

NAME	MEANING
`var`	always mutable
`ref`	inherits the mutability of the value origin it refers to

NON-OWNING VIEWS

A *view* is a non-owning window into a buffer someone else owns.

```
var data: List[String] = ["a", "b", "c", "d", "e"]
var s = data[1:3] # a Span view, no copy: [b, c]
s[0] = "X" # writes through: data is [a, X, c, d,
e]

var text = "Hello, World!"
var hi = text[codepoint=0:5] # a StringSpan view:
"Hello"
```

`Span` views contiguous elements; `StringSpan` views UTF-8 text.

Like `ref`, a view carries an origin, so the compiler keeps the source alive and tracks whether the view stays valid.

OWNERSHIP TRANSFER AND COPIES

TRANSFER OWNERSHIP WITH ^

```
def exclaim(var s: String):
    s += "!"
    print(s)

var g = "Hello"
exclaim(g) # implicit copy: g still usable
exclaim(g^) # transfer: g uninitialized
# print(g) # error: used after transfer
```

The `var` argument takes ownership of the original only with a transfer `^`; a plain call implicitly copies (`String` is `ImplicitlyCopyable`), so `g` stays usable. Either value, the copy or the transferred original, ends its lifetime after the `print()` (its last use). The same `^` drains a collection in a loop: `for var x in items^` transfers each element value out.

CALL SITES: PASSING VALUES

YOU WRITE	INTO A VAR ARG
`f(x)`	implicit copy (`ImplicitlyCopyable` only)
`f(x.copy())`	explicit copy
`f(x^)`	transfer; x uninitialized after

A borrowing argument (`imm`, `mut`, `ref`) has no `^` lever: you write `f(x)`, and it views the value in place.

CAPTURING VALUES INTO CLOSURES

CAPTURES

```
def peek() {imm n} -> Int: ... # borrow, read-only
def bump() {mut n}: ... # borrow, writes back
def keep() {var s} -> String: ... # own a copy
def take() {var t^} -> String: ... # own the
original
```

A closure's capture list supports `imm`, `mut`, `ref`, and `var`; `var name^` transfers the captured value from the outer binding.

STRUCT METHODS AND ARGUMENT CONVENTIONS

ARGUMENT CONVENTIONS ON SELF IN METHODS

CONVENTION	MEANING
`self`	imm (immutable)
`mut self`	modify the instance
`out self`	build it (in `__init__()`)
`deinit self`	destroy the instance
`ref self`	parametric mutability

ARGUMENT CONVENTIONS: THE DECISION TABLE

CONVENTION	OWNS IT?	MUTABLE?	CALLER KEEPS IT?	RE FO WI
`(imm)`	no	no	yes	rea va wi ch (th de
`mut`	no	yes	yes	ch th va pla
`var`	yes (own copy)	yes	yes, unless ^	yo loc mu co
`out`	yes (becomes the value)	yes	it is the result	ret by ins >
`deinit`	yes (consumes)	yes	no	de an so mo
`ref`	no (refers)	parametric	yes	ret or a r wi ori

A convention sits before the argument name: `def f(mut x: Int)`. With no convention, an argument is a read-only borrow: a view into a value you don't own. `mut` makes it a writable view.

LIFECYCLE

LIFECYCLE METHODS AND OWNERSHIP

CORE LIFECYCLE METHODS FOR OWNERSHIP MANAGEMENT

METHOD	MEANING
`__init__(out self, …)`	construct
`__init__(out self, *, copy: Self)`	copy
`__init__(out self, *, deinit move: Self)`	move
`__deinit__(deinit self)`	destroy

Copy, move, and destructors can't raise. The var assignment table shows which one each assignment runs.

OPTING OUT OF MOVES AND DESTRUCTION

```
struct Handle(not Movable): ... # pinned: can't
move

struct Lease(
    not Deinitable else "call 'release()' to return
the lease"
):
    def release(deinit self): ... # the only way to
end a Lease
```

By default a value can move, and it's destroyed at its last use.

`not Movable` pins a value in place.

`not Deinitable` makes it linear: the compiler rejects any path that drops it, and shows your reason.

RETURNS: HANDING A VALUE OUT

YOU WRITE	WHAT IT DOES
`return x`	copy out (when `ImplicitlyCopyable`)
`return x.copy()`	copy out
`return x^`	transfer out
`-> T`	return a value
`-> ref[origin] T`	return a reference

Like a var assignment, `return x` copies; when `x` is known to be at its last use in a given scope, the compiler transfers it instead of copying it.

No `-> T^`: the `^` goes on the returned value in `return x^`, not on the return type.